1 Survey Design and Methodology

1.1 Theoretical Framework

The survey instrument is grounded in the Unified Theory of Acceptance and Use of Technology (UTAUT). UTAUT synthesises eight prior technology acceptance models into four core behavioural predictors: *Performance Expectancy* (PE), the degree to which a developer believes metamorphic testing will improve their testing outcomes; *Effort Expectancy* (EE), the perceived ease with which MT can be learned and applied; *Social Influence* (SI), the extent to which peers, colleagues, or the broader community are perceived to endorse MT; and *Facilitating Conditions* (FC), the degree to which organisational or technical infrastructure supports MT adoption. These four constructs are modelled as predictors of *Behavioural Intention* (BI) to adopt MT, which in turn is a predictor of actual use. Because many respondents are expected to have no prior experience with MT, the survey also measures *Awareness* as a precondition that must be satisfied before the UTAUT constructs become operative. The overall model is illustrated in Figure 1.

Awareness $\longrightarrow$ PE + EE + SI + FC $\longrightarrow$ **Behavioural Intention** $\longrightarrow$ **Actual Use**

Figure 1: Structural model underlying the survey instrument. Awareness acts as a prerequisite gate: respondents with no prior exposure to MT receive an introductory definition before proceeding to the UTAUT items.

1.2 Survey Instrument Design

The instrument is structured into five sections. Section 1 collects demographic and professional background data, including programming experience in years, primary development domain, employment status (industry, academia, or student), company size, and primary programming languages. These variables serve both as descriptors of the sample and as moderators in the regression analyses.

Section 2 establishes MT awareness and prior experience. Respondents first indicate whether they have heard of metamorphic testing before taking the survey. Those who have not are shown a brief, standardised definition of MT and MRs before continuing; those who have are asked about the depth and recency of their experience. This branching structure allows awareness to be treated as an antecedent variable in the analysis, rather than conflating it with the attitudinal constructs that follow.

Sections 3 through 5 contain the core UTAUT scale items. Each construct is measured using between four and six Likert-scale items rated on a seven-point agreement scale ranging from *Strongly Disagree* (1) to *Strongly Agree* (7). The specific items were adapted from the validated UTAUT instrument, with wording adjusted to fit the MT domain. Table 1 lists each construct, its operational definition in the MT context, and the number of scale items used.

Table 1: Survey constructs, definitions, and measurement approach.

Construct	Operational Definition (MT context)	Items
Awareness (AW)	Prior exposure to and knowledge of metamorphic testing as a technique	2
Performance Expectancy (PE)	Belief that MT would improve testing quality or fault detection relative to current practice	5
Effort Expectancy (EE)	Perceived difficulty of learning to write metamorphic relations and integrate MT into a test suite	6
Social Influence (SI)	Perceived endorsement of MT by colleagues, managers, or the development community	4
Facilitating Conditions (FC)	Perceived availability of tools, libraries, documentation, and organisational support for MT	4
Behavioural Intention (BI)	Self-reported likelihood of adopting or continuing to use MT in the next 12 months	3
Adoption Behaviour (AB)	Current or past use of MT in professional or personal projects	3

Section 6 concludes with two open-response items soliciting free-text explanations of (1) the respondent's primary reason for not adopting MT, if applicable, and (2) any barriers or facilitators not captured by the closed items. These responses are analysed using inductive thematic coding after data collection.

1.3 Validity, Pilot Testing, and Ethics

Content validity of the scale items was evaluated by the project supervisors prior to deployment. A pilot administration of 12 participants drawn from the University of Auckland's School of Engineering verified item clarity, estimated completion time (approximately 12 minutes), and confirmed correct Qualtrics branching logic. The pilot responses were not included in the final dataset. The survey received ethical approval from the University of Auckland Human Participants Ethics Committee (UAHPEC) prior to data collection. Participation is anonymous and voluntary, with no identifiable data retained.

1.4 Sampling and Distribution

The target population is broadly defined as software developers with at least some programming experience, with no restriction on domain, employment type, or familiarity with MT. To reach a diverse cross-section of this population, the survey is distributed through multiple channels simultaneously: (1) Reddit communities for software developers and software testers; (2) LinkedIn posts targeting software engineering practitioners; (3) GitHub Discussions within repositories identified in the mining component as having explicit MT signals; and (4) direct email contact with maintainers of OSS projects. No minimum sample size was pre-registered, but a target of at least

80 complete responses is considered sufficient for regression-based analysis of the UTAUT model at the construct level.

1.5 Planned Analysis

Composite scores for each UTAUT construct are computed as the mean of their Likert items after verifying internal consistency (Cronbach's $\alpha \geq 0.70$). Confirmatory factor analysis (CFA) assesses discriminant and convergent validity before structural modelling. The model in Figure 1 is then estimated via OLS regression (BI onto PE, EE, SI, FC; actual adoption onto BI), with moderation tests for domain, experience level, and company size on the PE→BI and EE→BI paths. Open-response items are analysed using inductive thematic coding.

2 Repository Mining Pipeline

2.1 Design Rationale

The mining component is designed to produce a direct, conservative measure of *deliberate* MT adoption in open-source software, rather than inferring implicit MT-like behaviour from structural code patterns. This design choice is motivated by the methodological limitation of prior work: tools such as MR-Scout detect relational test patterns based on code structure regardless of whether the developer was aware of MT, which conflates latent capacity with deliberate adoption. Searching instead for explicit linguistic signals of MT awareness (the term "metamorphic" appearing in test files, comments, commit messages, and documentation) yields a measure more tightly coupled to the behaviour the study aims to explain.

The mining pipeline is implemented in Python and is structured as three sequential phases. Figure 2 provides an overview of the data flow.



Figure 2: Overview of the four-phase repository mining pipeline.

2.2 Phase 1: Search

Phase 1 queries GitHub across five search surfaces: repository metadata (names, descriptions, topics, README), code files (all files on the default branch), issues and pull requests, commit messages, and GitHub Discussions (via the GraphQL API). The keyword set covers canonical, plural, and singular forms of core MT terminology (`"metamorphic testing"`, `"metamorphic relation"`, `"metamorphic tests"`, `"metamorphic relations"`) plus underscore code-style variants. The search is deliberately broad: a Phase 1 hit means the keyword appeared somewhere in the indexed surface, not that MT is actively practised. Precision is deferred to Phases 2 and 3.

GitHub imposes a hard cap of 1,000 results per query. To achieve near-complete coverage, the tool implements adaptive date-range slicing: when `total_count` exceeds 900, the date range is bisected recursively to a maximum depth of six (up to 64 sub-windows per query). Rate-limit delays

of 2.2 s (standard endpoints) and 7.0 s (code search, a separate API bucket) prevent HTTP 429 errors. All results across overlapping queries and date slices are deduplicated by URL. Outputs are six JSON files in `results/raw/` plus a `summary.json` of per-query hit counts.

2.3 Phase 2: Clone and Filter

Phase 2 takes the repository candidates from Phase 1 and applies a set of criteria to identify repositories suitable for local code analysis. The current filtering criteria exclude repositories that are forks, exceed a maximum size of 200 MB, or fall below a configurable star threshold. Repositories passing these filters are sorted by star count in descending order and shallow-cloned (depth 1, latest commit only) into a local directory using GitPython. Shallow cloning limits disk usage while retaining all current files in the working tree, which is sufficient for the static pattern analysis in Phase 3. A manifest file (`results/cloned/manifest.json`) records the clone status (`cloned`, `skipped`, or `error`) and metadata for each candidate.

2.4 Phase 3: Local Analysis and Classification

Phase 3 walks every file in each cloned repository and applies a set of regular expression patterns to detect explicit signals of MT usage. The patterns are grouped into three categories, as shown in Table 2.

Table 2: Explicit MT detection patterns used in Phase 3 analysis. All patterns operate on the raw source file; lemmatised comment patterns additionally operate on a normalised version of extracted comment and docstring text (see Section 2.5).

Pattern Category	**Examples**
Library imports / identifiers	`import metamorphic`, `from metamorphic`, `MetamorphicTest`, `MetamorphicRelation`, `@metamorphic` (decorator)
String and comment references	`"metamorphic testing[s]?"`, `"metamorphic relation[s]?"` in string literals or inline comments (`#` or `//`)
Lemmatised comment patterns	Morphological variants (e.g., *metamorphically*, inflected forms) matched against NLTK-lemmatised comment/docstring tokens
Language-specific conventions	Java void method names matching `testMR*()` (following MT literature naming convention); Java class name `MetamorphicTesting`

All patterns are applied case-insensitively. For each match, the tool records the repository, file path, line number, matching pattern identifier, and a one-line code snippet with one line of surrounding context. After walking all files in a repository, an aggregate score is computed and the repository is assigned to one of five classification tiers:

- `strong_explicit`: ≥ 3 explicit pattern matches. Multiple unambiguous signals of deliberate MT adoption are present, such as library imports, class usage, and comment references occurring together across the repository.

- `weak_explicit`: 1–2 explicit matches. At least one direct signal is present, but it is isolated; adoption may be exploratory or confined to a single file.
- `none`: No explicit signals found in source code. The repository appeared in search results due to the keyword occurring in a README, commit message, or issue body, but no MT patterns were identified in code.

The scoring formula assigns 3 points per explicit hit, capped at 100, to allow for ranking within tiers. The analysis default runs in `-only explicit` mode, focusing exclusively on deliberate MT signals and avoiding the noise introduced by broad structural heuristics.

2.5 Lemmatization and Resolved Limitations

Raw regex patterns match only the exact string forms present in source files. Morphological variants in comments and documentation — such as *metamorphically* or inflected noun forms — would otherwise constitute false negatives. To address this, Phase 3 now applies NLTK-based lemmatization as a preprocessing step for comment and docstring content: comment lines and docstring blocks are extracted, their tokens are lemmatised via `nltk.stem.WordNetLemmatizer`, and a secondary set of comment patterns is matched against the normalised string. Findings from this lemmatised pass are tagged as explicit with a distinguishing snippet label, and a de-duplication guard prevents double-counting when the raw pattern already matched. Structural code patterns (imports, class names, decorators) retain exact-match regexes since lemmatization is inappropriate for source-level identifiers.

Several earlier limitations have also been resolved. Library-specific code queries referencing non-existent packages (`pytest-metamorphic`, `hypothesis_metamorphic`, `mt_testing`, `org.metamorphictesting`) have been removed from both the search configuration and the local analysis patterns, eliminating a significant source of false positives. The `oracle` keyword has been removed because it is too general to serve as an MT-specific signal. Implicit pattern searching — structural heuristics that infer MT-like behaviour from test code shape rather than explicit terminology — is out of scope for this study; the `IMPLICIT_PATTERNS` list is now empty and the analysis default is `-only explicit`. Future work may explore a targeted search for tests referencing both a *source test case* and *follow-up test case* within the same function, though this is an advanced extension beyond the current scope.

2.6 Phase 4: Visualisation

Phase 4 (`plot.py`) is a standalone step that reads the outputs of Phases 1 and 3 and produces eight publication-quality figures in `results/plots/` using `matplotlib`, `seaborn`, and `pandas`. Each plot function checks for its required input file and skips gracefully if absent, allowing the phase to run incrementally. The eight outputs are: per-query hit counts (horizontal bar); classification tier distribution; yearly trend of explicit MT-adopting repositories by creation date; language distribution of explicit MT repositories; scatter plot of star count versus explicit hit count; explicit pattern hit frequency; language–classification tier heatmap; and query-overlap distribution (how many repositories were found by more than one distinct search query). These plots directly support the RQ2 sub-questions and are designed for inclusion in the final thesis.

3 Integration of Survey and Mining Results

3.1 Cross-Method Triangulation

RQ3 asks whether developer beliefs about MT are consistent with the observed prevalence of MT in open-source repositories, and what any divergence reveals about adoption. Answering this question requires a structured integration of the survey and mining datasets, rather than treating each as independent.

The triangulation strategy proceeds as follows. The mining dataset provides, for each repository, a classification tier (from `strong_explicit` to `none`), the programming language, the repository domain as proxied by topics and description, and time-series data on when explicit MT adoption first appeared. These objective adoption indicators are compared against the distributional properties of the survey constructs to examine whether the patterns that developers self-report about MT perception are reflected in what the mining data shows about actual practice.

Three comparisons are planned. First, the survey's awareness distribution (proportion of respondents who had never heard of MT) is compared against the mining funnel (proportion of keyword-matching repositories that contain any explicit MT signal in source code), quantifying the awareness-to-adoption gap. Second, domain-level breakdowns in the survey are compared against the domain distribution of `strong_explicit` repositories to test whether domain is a genuine moderator of adoption. Third, Effort Expectancy scores are compared between survey respondents who have adopted MT and those who have not, and cross-referenced against the star-count and maturity profile of mining-identified adopters, to test whether perceived effort and actual project characteristics are consistent.

3.2 Limitations of the Integration

The primary limitation of this triangulation approach is that the survey and mining data describe potentially different populations. Survey respondents are self-selected, while mined repositories are determined by GitHub's search index. There is no guarantee that the repositories a respondent maintains appear in the mined corpus, or that the survey respondents are the same developers who wrote the code the mining component examines. The integration is therefore at the level of population-level patterns rather than individual-level linkage. This is a structural constraint of the method combination, and the analysis treats the two data sources as complementary lenses on the same phenomenon rather than as a matched dataset.